

Part I — The committer on its own

Setup: the committer machines

Nineteen machines, each 64 cores and 156 GiB, running three signature verifiers, one coordinator, one sidecar, one load generator, six validator-committers, twelve YugabyteDB tablet servers and three YugabyteDB masters. The six validator-committers and the three masters sit on nine of the twelve database machines, and no machine runs both a validator-committer and a master.

Storage is two 969 GiB XFS volumes per machine, `noatime`, mounted `/data1` and `/data2`. Each database node passes YugabyteDB both volumes as separate data directories (`--fs_data_dirs`, one path per volume) rather than a stripe. The sidecar's ledger uses `/data1` alone.

Transactions carry two read-write operations over 32 B keys, 262 B serialized, in 10,000-transaction blocks, and each transaction slot takes a fresh key, so there are no read-write conflicts. Blocks come from the load generator's embedded mock ordering service, which cuts and signs them itself and serves them to the sidecar over the Atomic Broadcast API, so no ordering service is in the path.

Signatures are ECDSA P-256, matching Part II. *The figures in this part were measured under Ed25519* and are being re-measured; until they are, the two parts differ in signature scheme as well as in ordering. The reason for the change, and for the Ed25519 that preceded it, is below.

A measured point is the highest offered rate that arrived in full, committed within 2%, kept p99 under 1 s and held a flat in-flight count, confirmed by a 300 s hold on a fresh deployment.

Why this setup

The shape is not the paper's. The published committer deployment is nine validator-committers on nine database nodes, dual 48-core servers with 64 GB. What follows therefore compares deployments, not matched hardware.

Both volumes, not a stripe, so YugabyteDB sees the disk boundary and places data across it itself. This also has to be stated because it is what a wipe must cover: the data directories are siblings of the deployment directory, not children, so clearing the deployment directory leaves the database intact — which silently invalidated every earlier bring-up of the arm in Part II.

ECDSA rather than Ed25519, so the two arms differ only in whether an ordering service is present. This was Ed25519 until the generator stopped being the constraint: `crypto/ecdsa` signs hedged, building an HMAC-SHA-512 DRBG per signature, which a CPU profile at 193,000 tps put at 8.11% of process CPU against 8.80% for the elliptic-curve multiplication it feeds — nonce setup costing more than the work it exists to serve. Deriving the nonce directly is worth $2.06\times$ at 32 cores, about 709,000 signatures a second, above the ceilings below. Entropy was never the mechanism (`GetRandom` was 0.54%), and two earlier explanations — `getrandom` contention, then unspecific allocation — were both wrong.

A conflict-free workload measures the pipeline's ceiling rather than its conflict behaviour, at the cost of a state database and a ledger that grow with every transaction. Conflicts are then introduced deliberately as their own variable.

Queue flatness is part of the definition because a rate can be committed in full out of a growing queue, which reports the queue's drain time as latency.

Throughput and tail latency

Throughput falls with transaction size (Fig. 1), 604,545 tps at one read-write operation to 274,364 at four: 55% against the published 41%. The mechanism is not the published one, which is lock contention in the coordinator's dependency graph. Here both lock-wait histograms have a count rate of exactly zero, and the graph's cost is per key instead — about 420 ns each, flat from one key per transaction to eight.

Invalid signatures are close to free but never profitable: 558,364 tps with none, then 517,455, 519,455 and 518,727 at 10%, 20% and 30% — flat within 0.4%, not the published 10% *gain*. Driven instead at one common 559,872 tps, all three deliver it while the tail grows with the invalid share (1.1, 5.0 and 7.5 s): rejection is cheap in throughput and expensive in latency.

Double spends do not reproduce, and a deployment setting is why. At the 120-way tablet pre-split this cluster uses, 0% holds 518,000 tps and *no* rate qualifies at 5% or above — about 20,000 tps at tens of seconds with every machine under 10% CPU, so neither capacity nor a queue. At the default split the same shapes hold: 41,273, 30,000 and 40,909 tps at 10%, 20% and 30% with 334 ms to 533 ms p99, flat rather than declining — while costing the conflict-free case $1.8\times$ (518,000 against 294,893). The split is therefore a workload-dependent trade, not a tuning win. Narrowing the graph's chunk width refuted multi-key batching as the mechanism, leaving why 120 tablets collapse unresolved.

What latency costs at a given throughput

At 10,000-transaction blocks the median has a floor near 125 ms that does not fall with the rate, since nothing in a block moves until the block is cut (Fig. 2). At 500 transactions a block the floor is 40 ms, and 380,100 tps holds at 125 ms median and 208 ms p99 — the median large blocks give at 10,000 tps, at thirty-eight times the rate. The top of that curve measures the generator, whose block preparation caps it near 850 blocks a second (425,000 tps at this size; 450,000 offered delivered 432,309), so the committer's own ceiling at 500 transactions is not yet known.

Part II — End to end, with a real ordering service

The second arm puts a real Arma ordering service in the path. The committer tier is **unchanged from Part I** — the same nineteen machines, the same components, the same tuning — so a difference between the two arms is attributable to ordering.

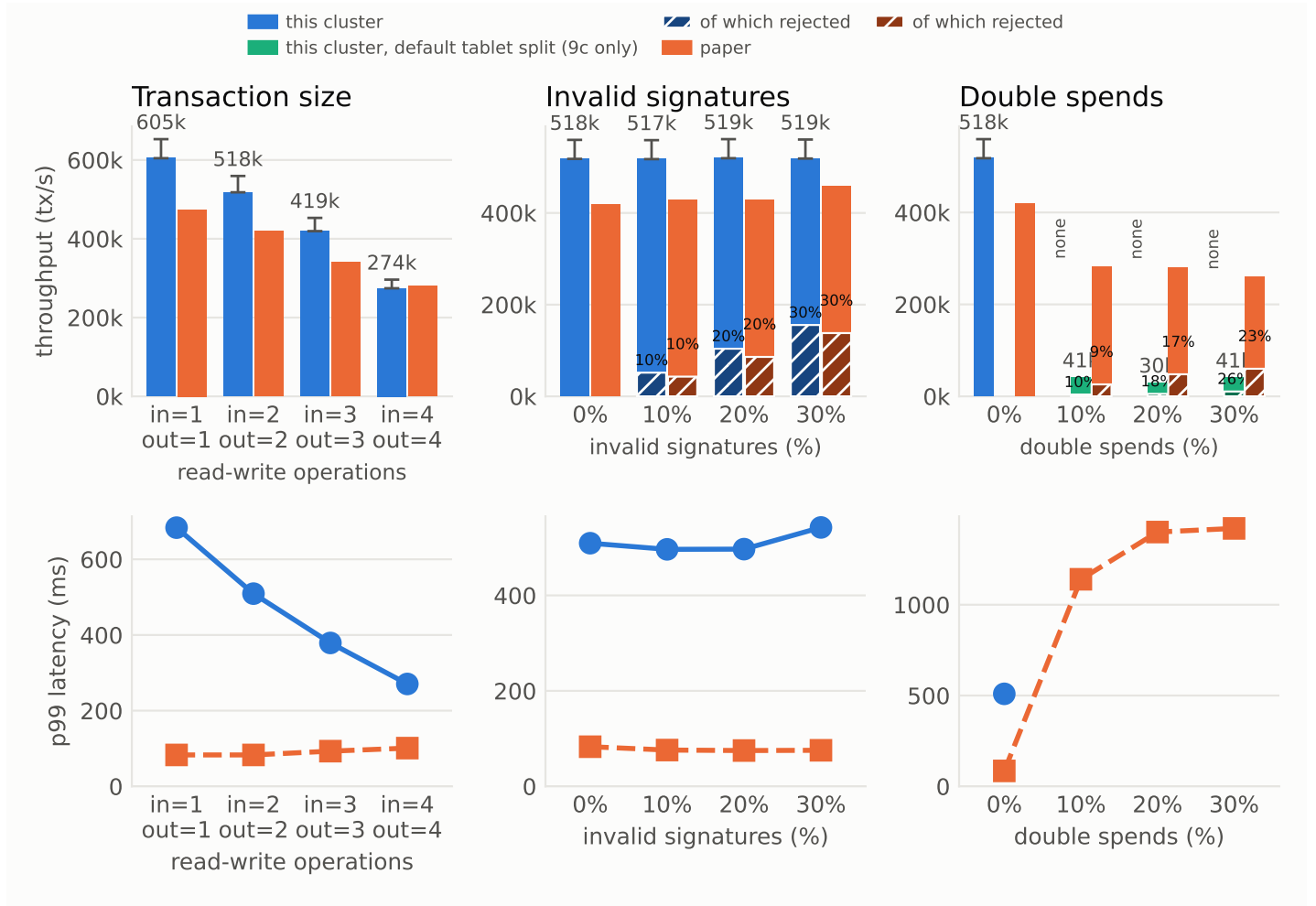


Figure 1: Committer throughput and 99th-percentile latency against transaction size, invalid-signature share and double-spend share, beside the published figures. Each bar is the highest rate held for 300 s with p99 under one second, no queue growth, and the offered rate arriving. Throughput counts transactions *finished*, committed plus rejected; the hatched part of a bar is the rejected share, labelled as a percentage, so the two experiments are compared at matched reject rates rather than at matched labels. Whiskers are the search’s 8% step, one-sided because a knee is a lower bound. *none* marks a condition where no offered rate qualified — the absence is the result. The published series is Figure 9 of the paper: its endpoints as quoted in the text, intermediate points read off the plots at 400 dpi.

Setup: the ordering machines

Twenty further machines, each 32 cores and 78 GiB, holding four parties of a router, a consenter, an assembler and one batcher per shard. Routers, consenters and assemblers take a machine each, twelve in all; the batchers share the remaining eight. Two configurations are compared:

- **Four shards.** Sixteen batchers, two to a machine, each with a volume to itself (`/data1` and `/data2`).
- **Eight shards.** Thirty-two batchers, four to a machine, two sharing each volume.

Storage is two 485 GiB XFS volumes per machine, mounted as on the committer machines. Routers, consenters and assemblers write to `/data1`; the batchers are distributed across both volumes as above.

Blocks are cut by the batchers, so the block-size knob on this arm is the shared configuration’s `Batching.BatchSize.MaxMessageCount`, not a load-generator setting, and `BatchCreationTimeout` is 500 ms.

Signatures are ECDSA P-256, as in Part I, and here the policy derives from a real MSP identity so no other scheme is available. The namespace is created by `fxconfig` before load is applied, because the transaction that creates it endorses against an MSP rule that the load generator has no material for.

Why this setup

Two batchers to a machine is measured rather than assumed. At two shards the service capped near 158,000 tps *per shard* whatever the batch size or decision interval, while batcher processes sat at 15% to 20% of a 32-core machine — so the constraint is per shard and the spare capacity for a second was already on the box.

Both shard counts are measured because that reading predicts more shards buy throughput, and the two configurations separate the two things a shard needs: a core budget and a disk. Four shards give each co-located batcher its own volume; eight give twice the shards but make two share one.

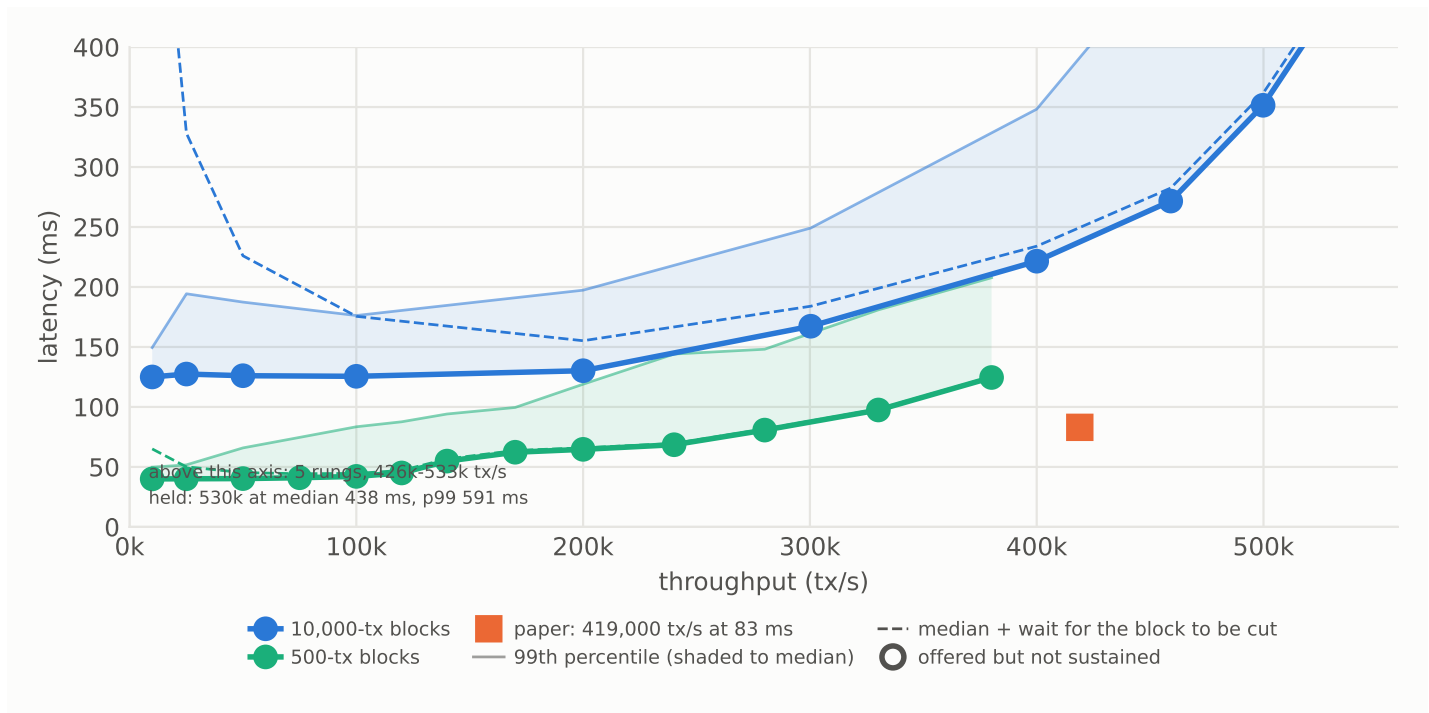


Figure 2: Latency against throughput at two block sizes. The thick line is the median and the thin line above it the 99th percentile, shaded between; the dashed line adds the mean wait for a block to be cut, which the measurement itself excludes. Hollow markers are rates offered but not sustained. The axis is clipped at 400 ms because the region worth reading is 40 ms to 400 ms; what the clip cuts off is named in the corner, and recorded per point in `figures-table.md`.

These machines are half a committer machine — half the cores, half the memory and, per Table 2, half the disk bandwidth — so a ceiling measured on this arm belongs to these machines before it belongs to ordering.

Shard count and storage

Both configurations sustain the same throughput and hit the same knee: 499,091 tps at four shards against 500,000 at eight, 0.2 % apart, and neither sustains 550,000 (Fig. 3). Four shards is the better configuration on every other axis — p99 lower by 100 ms to 250 ms at every matched rate from 100,000 tps up (658 ms against 754 ms at the knee), fewer processes, and a volume per batcher rather than two sharing one.

Doubling the shards buys nothing because ordering is not what limits this arm. Through 400,000 tps the busiest machine is a router; from 450,000 on it is a validator–committer at 78 % to 82 %, on both configurations and at low fill as well as high. Extra shards add capacity to the tier that is not binding.

What the knee itself is cannot be attributed so cleanly, and the honest statement is narrower than “committer-bound”. At 500,000 tps no machine anywhere exceeds 82 %: the busiest validator–committer is at 81 % and *the load generator is at 77 %*, four points behind it. So the ceiling is neither a CPU wall on any tier nor separable by this experiment — the two candidates are within each other’s margin, and a ladder on one deployment cannot tell them apart. Naming the busiest host names a correlate, not a cause.

They also cost latency, for a reason the block size makes plain. At a given total rate each of eight shards sees half the per-shard rate that four do, so batches take

twice as long to reach 10,000 transactions and the 500 ms `BatchCreationTimeout` cuts them early over twice as wide a range of rates. The eight-shard curve shows exactly that shape: p99 rises to 793 ms at 150,000 tps, then falls monotonically — 687, 614, 591 ms — as the rate grows enough to fill a batch before the timer fires.

Is the knee the rate, or a full database?

It is the rate. This needed testing rather than asserting, because each ladder climbs on one deployment, so offered rate and table fill rise together: over the eight-shard ladder the database grew from 9.4 million rows to 1.31 billion, free disk fell from 513 GB to 62 GB, and database commit latency went from 18 ms to 232 ms. On that evidence alone a knee at 500,000 tps is as consistent with a full table as with the rate.

Both rates were therefore re-measured on fresh deployments, at low and matched fill. Four attempts now exist at 550,000 tps across a 140× range of fill, and *none* sustains it; three exist at 500,000 and three of those pass. The boundary does not move with fill, so it is a rate.

What fill does change is the tail, and only near the knee. At 550,000 tps the same eight-shard configuration gives p99 1.43 s on a fresh database against 4.9 s at 1.1 billion rows, and how much is delivered tracks fill the same way — 546,364, 543,818, 532,909 and 497,603 tps in order of increasing fill. Below the knee the effect is a few percent; above it, queues form and fill multiplies what they cost.

The re-measurement also refines which configuration to prefer, in the direction the batching mechanism predicts: extra shards pay only once fewer have run out of capacity. Up

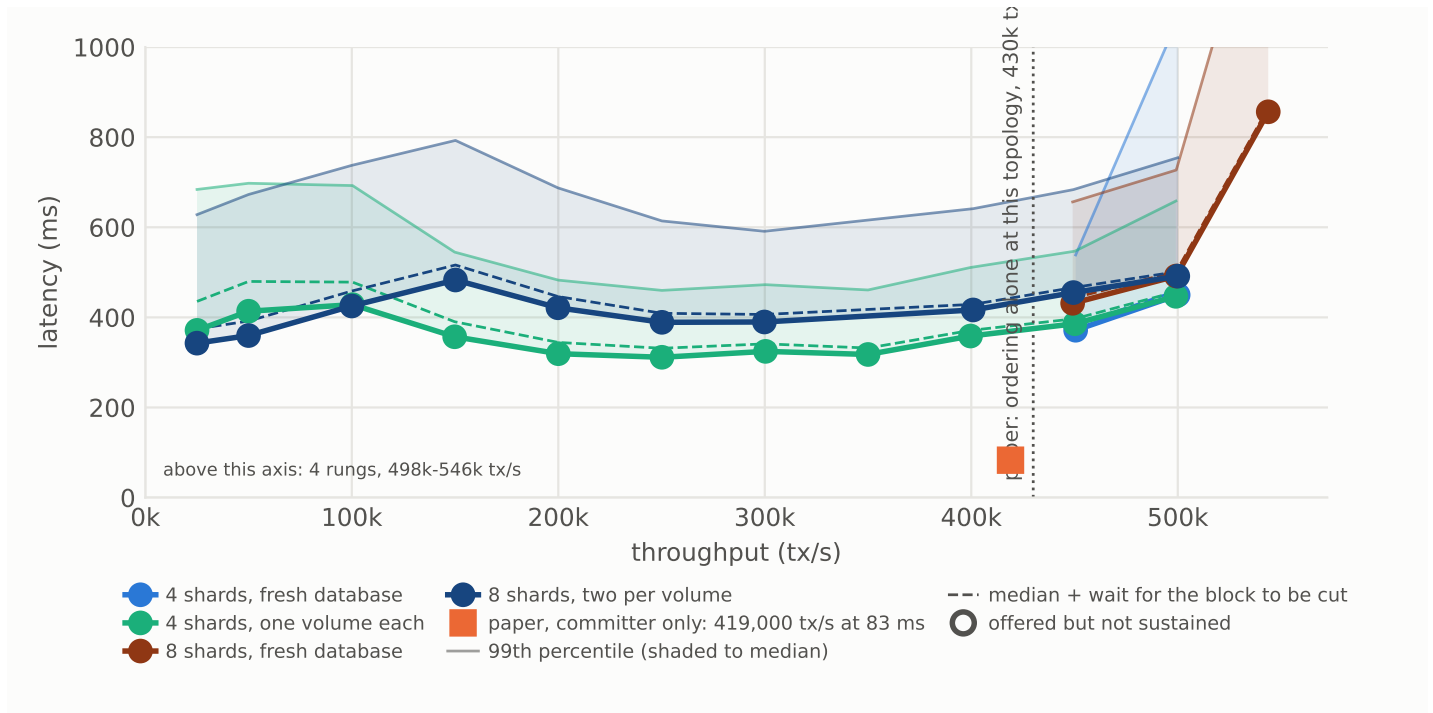


Figure 3: Latency against throughput end to end, with a real ordering service in the path, by shard count and batcher storage layout. Marks as in Fig. 2. The axis is clipped at 1,000 ms rather than 400 ms, because here the batchers cut the blocks with a 500 ms `BatchCreationTimeout` and the median floor sits above 400 ms. The dotted line is the published ordering-alone figure for this topology.

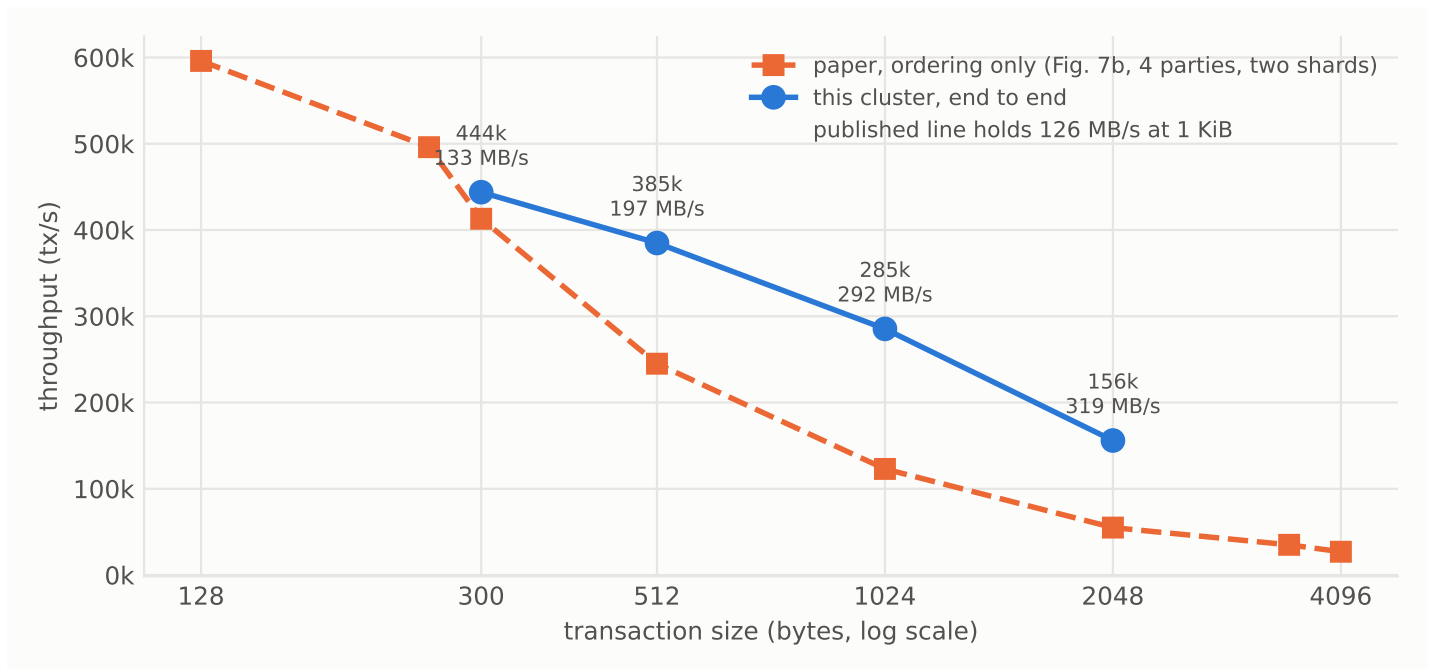


Figure 4: Throughput against transaction size, end to end, beside the published Figure 7b. The published line measures *ordering alone* at two shards; this one measures the complete pipeline at four, so the comparison is of shape rather than of values. Each point is annotated with the byte rate it implies, which is where the two diverge: the published line is flat near 126 MB/s^{-1} and this one rises from 133 to 319 MB/s^{-1} .

to 500,000 tps four shards is better, as above. At 550,000 eight shards degrades far more gracefully — 1.43s against 7.5s at matched low fill — because there the constraint is capacity rather than batch fill. Neither sustains it under a one-second bound.

Latency against throughput, ordering included

End to end, ordering included, this deployment sustains 499,091 tps at 658 ms p99, with every offered rate up to that arriving in full and no aborted transactions at any rung. Two

comparisons place it. The published work has no end-to-end figure, but its ordering service alone reaches 430,000 tps at four parties and four shards — so the complete pipeline here exceeds the published figure for one part of it. And the same committer without an ordering service reaches 604,545 tps, so real ordering costs about 17% of the committer’s own ceiling rather than an order of magnitude.

Batch size is the latency–throughput knob here, and it trades one for the other almost exactly. Cutting the batch from 10,000 transactions to 500 removes most of the latency at every rate the two share — p99 197 ms against 693 ms at 100,000 tps, 286 ms against 460 ms at 250,000, and mean latency a third of what large batches give. That is the predicted effect and its direction confirms the mechanism: a batch a twentieth the size fills twenty times sooner, so the 500 ms `BatchCreationTimeout` stops being what cuts it.

What it costs is the ceiling, and the reason is exact. The block rate tracks the offered rate precisely — 200 blocks a second at 100,000 tps, 400 at 200,000, 500 at 250,000 — and between 500 and 600 blocks a second the arm stops working. 250,000 tps is therefore not a transaction limit at all: it is 500×500 , a *block-rate* ceiling near 500 a second. At 10,000 transactions a batch the same block rate would carry five million transactions a second, which is why that configuration is limited by transactions instead and reaches 499,091.

So the two configurations are bounded by different things, and the choice is a real trade rather than a tuning win: small batches halve the latency and halve the ceiling.

The failure at 300,000 tps is worth recording because it is not graceful. Throughput does not level off, it collapses — to 773 transactions and 2 blocks a second, with every machine under 2% CPU and the disk untouched. The load generator then begins resubmitting transactions it has already had committed, and about a tenth of what does arrive comes back `REJECTED_DUPLICATE_TX_ID`, so the retry amplifies the stall it is responding to. A deployment run just past this ceiling does not degrade, it stops.

Transaction size

Throughput falls with transaction size but far more slowly than the published sweep, and the byte rate is what shows why (Fig. 4). The published measurement is bandwidth-bound past 300B: each of its points sits within 10% of 120 MB s^{-1} , so its rate is simply bytes per second divided by size. This deployment is not bandwidth-bound anywhere in the range. Its byte rate *rises* with size, from 133 MB s^{-1} at 300 B to 319 MB s^{-1} at 2 KiB, because what it pays per transaction dominates what it pays per byte: halving the transaction count at twice the size is a net gain. Throughput therefore falls about $0.76\times$ per doubling rather than the $0.5\times$ a bandwidth-bound service gives.

Where that stops is measurable, and it is the assembler’s disk. At 4 KiB throughput plateaus near 120,000 tps — 492 MB s^{-1} of transaction payload — with no machine anywhere above 54% CPU, so it is not a processing limit. The assemblers at that point are writing 527 MB s^{-1} to 546 MB s^{-1} to their data volume, against the 529 MiB/s sequential write Table 2 measures on these machines: the

Size	Throughput	Byte rate	Evidence
300 B	443,818	133 MB/s	hold, conservative
512 B	384,909	197 MB/s	hold, lower bound
1 KiB	285,353	292 MB/s	probe; hold had no disk
2 KiB	155,936	319 MB/s	confirmed hold
4 KiB	100,212	410 MB/s	probe; disk-bound

Table 1: What each size point rests on, since they do not all meet the same bar — and where a hold exists it is quoted in preference to a higher probe, because a 90s probe is routinely optimistic: 300 B passed 518,182 on a probe and then failed its 300s hold at that rate outright. *Confirmed hold* is the full criterion. *Conservative*: the 300 B point was measured before the driver drained between attempts, so its confirmation stepped down against a queue left by the rate above it and the figure is a floor. *Lower bound*: the 512 B search met every step of its climb and never bracketed a knee, so this is the highest rate tried rather than the highest sustainable. *Hold blocked*: at 1 KiB the knee was bracketed by probes, but each assembler writes the complete block stream, so at 292 MB s^{-1} its 485 GB volume filled before the 300s hold could run — and at 4 KiB the attempt filled it completely, to 4.9 MB free, stopping the ordering service. A hold is now refused unless there is room for what it will write. The first two points are being re-measured.

device is saturated. Each of the four assemblers writes the *complete* block stream, where a batcher writes only its own shard’s share, so the assembler is where a byte rate first meets a disk. At 2 KiB the same figure is 319 MB s^{-1} , three fifths of that ceiling, which is why that point holds cleanly and this one does not.

So the sweep has two regimes rather than one. Below 2 KiB the arm is limited per transaction and the byte rate climbs; at 4 KiB it is limited per byte by one component’s disk, and the published bandwidth-bound model finally applies here too — at four times the bandwidth, on a volume half the size.

The consequence for the comparison is that only the direction is shared. Both lines fall, and this one is above the published line at every size they have in common — but it is a whole pipeline against one of its parts, at four shards against two, so the gap is not a speedup and the shapes are not measuring the same limit.

Storage characterisation

Table 2 gives `fio 3.35` in the four patterns this deployment produces. The volumes are `virtio` devices reporting neither a model nor a meaningful rotational flag, so the backing media cannot be identified from inside the guest and is characterised by measurement instead.

Two things follow. Ordering machines have half the disk bandwidth as well as half the cores, at almost exactly a factor of two on every bandwidth row, at round figures, repeating to four significant figures across machines and volumes — these are provisioned caps, not device characteristics, so a ceiling on that arm must be read against the cap before it is attributed to ordering. And durability is the expensive direction: making one 4 KiB record durable with an `O_DSYNC` write costs about 400 μs , capping any per-record design near 2,400 operations a second, three orders of magnitude below the rates measured here. The ledger survives that only by

Pattern	Depth	Committer	Ordering
1 MiB seq. write	4	1,058 MiB/s	529 MiB/s
4 KiB O_DSYNC write	1	2,400 IOPS	3,249 IOPS
8 KiB rand. read	32	126,000 IOPS	67,689 IOPS
8 KiB rand. write	32	102,000 IOPS	51,165 IOPS

Table 2: `fiio 3.35, libaio, direct=1`, 20 s per pattern, run on a volume the deployment was not using at the time. The synchronous row is the exception to the pattern: the ordering disks are *faster* there (297 μ s against 395 μ s at the median) despite half the bandwidth. Committer figures are the range across a database node and the sidecar. A synchronous engine caps `fiio`’s queue depth at 1 while still reporting the depth asked for, so the deep rows use `libaio`.

batching — it appends a whole block and syncs every hundredth. Syncing alone is cheap by comparison, an `fdatasync` after a buffered write returning in about 0.08 ms: it is the write, not the barrier, that costs.

Threats to validity

The load generator is the instrument and twice became the limit: ECDSA signing, resolved by deriving the nonce directly, and block preparation, which caps the small-block curve in Fig. 2. It is close to being the limit a third time. At the end-to-end knee it runs at 77 % of its 64 cores against the busiest validator–committer’s 81 %, and on the 300 B size point at 80 % against 81 % — tied, within the noise. Every figure near 500,000 tps should therefore be read as a bound on the pair, generator and pipeline together, not on the pipeline alone. Separating them needs a second generator, and there is no spare machine for one without taking it from the committer. A knee is resolved to the search’s 8 % step, so it is a lower bound, drawn one-sided in Fig. 1. The baseline drifts 10 % between days, so only same-day comparisons are used.

One limit is stated in advance of the end-to-end data rather than after it. That arm measures a ladder on a single deployment, so rate and table fill rise together and the ladder cannot separate them: if its curve bends upward at the top, that is as consistent with a fuller database as with the offered rate. On the committer arm the question was settled by holding the rate fixed while fill grew through one 375 s window, where commit latency stayed flat — 142.2 ms to 143.0 ms across 68 % fill growth — but that test belongs to that arm and does not transfer. Until it is repeated here, an upward bend in the end-to-end curve is not attributable to rate. Each rung records the fill it ran against, so the question stays auditable rather than lost.